

# 26 年春季计算机系统基础期中考试

## 一、只缘身在此山中

（共 15 分）假设在一个 `int` 类型长度为 32 位的机器上运行程序，整数  $x$ 、 $y$ 、 $z$  都是随机生成的。

```
int x = random(); int y = random(); int z = random();
```

整数的溢出规则按课程约定进行处理。判断以下命题的正确性：

1. `x == (int)(float)x`  
A. 正确     B. 错误
2. `x >= 0 => -x <= 0`  
A. 正确     B. 错误
3. `(unsigned)x / 2 == (unsigned)x >> 1`  
A. 正确     B. 错误
4. `x > 0, y > 0 => x + y > 0`  
A. 正确     B. 错误
5. `x + y == x + z => y == z`  
A. 正确     B. 错误

## 二、任尔东西南北风

（共 15 分）在 Linux 操作系统中，文件权限的底层控制是通过高效的位运算来实现的。

系统使用一个 `size_t` 类型变量，用它的最低 3 个二进制位来表示用户对某文件的访问权限。这 3 位表示的权限为：

- 第 2 位（这 3 位里的最高位）代表读权限（`read`），对应二进制 `0b100`（即十进制 4）；
- 第 1 位（这 3 位里的中间位）代表写权限（`write`），对应二进制 `0b010`（即十进制 2）；
- 第 0 位（这 3 位里的最低位）代表执行权限（`execute`），对应二进制 `0b001`（即十进制 1）。

当前用户对某个文件的权限记录在 `size_t` 类型变量 `mode` 中。权限位可组合，例如，当 `mode = 5`（二进制 `0b101`）时，表示用户拥有读（4）和执行（1）权限，但没有写权限。

掩码是一个用于按位操作的二进制数值，用 1 表示“要操作/关心的位”，0 表示“不操作/忽略的位”。掩码记录在 `MODE_MASK` 枚举类变量 `mask` 中。`MODE_MASK` 枚举类包含：

- `0b001`（1）- 执行权限；
- `0b010`（2）- 写权限；
- `0b100`（4）- 读权限。

例如，当 `mask = 4`（二进制 `0b100`）时，表示关心的是读（4）权限。

请补全代码框架中的三个 C++ 函数，利用**位运算符**高效实现文件权限的**校验、添加和撤销**功能。

**校验：**参数 `mode` 是当前文件的权限状态变量，参数 `mask` 是需要校验的权限掩码。如果包含对应权限返回 `true`，否则返回 `false`。

**添加：**参数 `mode` 是当前文件的权限状态变量的引用，参数 `mask` 是需要添加的权限掩码。修改 `mode` 的值，为用户添加对应的权限。

**撤销：**参数 `mode` 是当前文件的权限状态变量的引用，参数 `mask` 是需要撤销的权限掩码。修改 `mode` 的值，为用户撤销对应的权限。

禁止使用：

- 内联汇编；
- 条件语句（`if` 或者 `?:`）、循环语句、分支语句、函数调用和宏调用；
- 加法、减法、左移、右移、乘法、除法和模运算；
- 相对比较运算（`==`、`!=`、`>`、`<`、`<=` 和 `>=`）；
- `MODE_MASK` 和 `size_t` 以外的数据类型；
- 逻辑运算（逻辑与 `&&`、或 `||`、非 `!`）。
- 常数。

仅允许：

- 位级运算（位级与 `&`、或 `|`、非 `~`）；

函数框架如下:

```
#include <stdio.h>

// 定义权限掩码枚举
enum MODE_MASK {
    EXE    = 0b001,  // 1: 可执行
    WRITE  = 0b010,  // 2: 可写
    READ   = 0b100   // 4: 可读
};

/**
 * @brief 校验用户是否拥有指定的权限
 * @param mode 当前文件的权限状态变量
 * @param mask 需要校验的权限掩码
 * @return 如果包含该权限返回 true, 否则返回 false
 */
bool check_permission(size_t mode, MODE_MASK mask) {
    return /*你需要填写的*/;
}

/**
 * @brief 为用户添加指定的权限
 * @param mode 当前文件的权限状态变量的引用, 注意是引用传值, 你需要直接修改原值
 * @param mask 需要添加的权限掩码
 */
void add_permission(size_t& mode, MODE_MASK mask) {
    mode = /*你需要填写的*/;
}

/**
 * @brief 撤销用户的指定权限, 若用户无指定权限, 则什么也不做
 * @param mode 当前文件的权限状态变量的引用, 注意是引用传值, 你需要直接修改原值
 * @param mask 需要撤销的权限掩码
 */
void revoke_permission(size_t& mode, MODE_MASK mask) {
    mode = /*你需要填写的*/;
}
```

### 三、天淡云闲今古同

(共 15 分) DeepSeek 在其模型训练和推理中广泛使用了 FP8 精度。NVIDIA H100 显卡支持两种 FP8 格式: E5M2 和 E4M3。其中, E5M2 有 1 位符号位, 5 位表示指数, 2 位表示尾数; E4M3 有 1 位符号位, 4 位表示指数, 3 位表示尾数; 均采用向偶数舍入的舍入方法, 其余与 IEEE 754 标准类似。

- (1) 将十进制数 5.5 表示为 E5M2 格式的 FP8 浮点数, 其位级编码为\_\_\_\_\_ (填写格式: 十六进制, 形如 0x12) (5 分)
- (2) 某 E4M3 格式的 FP8 浮点数, 位级编码为 0xCA, 其表示的数值为\_\_\_\_\_ (填写格式: 十进制整数或十进制小数, 形如 1234、-1234、1234.5678 或 -1234.5678, 若为有限小数则写全, 若为无限小数则保留小数点后 4 位) (5 分)
- (3) 现将机器精度定义为 1 与比 1 大的最小的可表示浮点数之间的差。记 E5M2 规格的 FP8 浮点数的机器精度为  $\epsilon_p$ , E4M3 规格的 FP8 浮点数的机器精度为  $\epsilon_q$ 。则  $\epsilon_p/\epsilon_q =$  \_\_\_\_\_ (填写格式: 十进制整数或十进制小数, 形如 1234、-1234、1234.5678 或 -1234.5678, 若为有限小数则写全, 若为无限小数则保留小数点后 4 位) (5 分)

### 四、道是无晴却有晴

(共 23 分) 考虑下面的 Union 声明:

```
union ele {
    struct {
        long (*f) (long);
        long y;
    } e1;
    struct {
        long x;
        union ele *next;
    } e2;
};
```

(1) 下面的字段偏移址是多少 (以字节为单位): (8 分)

- e1.f: \_\_\_\_\_
- e1.y: \_\_\_\_\_
- e2.x: \_\_\_\_\_
- e2.next: \_\_\_\_\_

(2) 这个结构体一共需要\_\_\_\_\_ 个字节。(2 分)

下面的函数 (省略了一些表达式) 对一个链表进行操作, 链表是以上述 Union 作为元素的:

```
void proc(union ele *up){
    up->___ = ____;
}
```

编译器为 `proc` 产生下面的汇编代码:

```
proc:
    pushq %rbx
    movq  8(%rdi), %rax
    movq  %rdi, %rbx
    movq  8(%rax), %rdi
    call  *(%rax)
    movq  %rax, (%rbx)
    popq  %rbx
    ret
```

(3) 在这些信息的基础上, 填写 `proc` 代码中缺失的表达式。(5 分)

提示: 有些联合引用的解释可以有歧义, 当你清楚引用指引到哪里的时候, 就能够澄清这些歧义。对内存的引用都是安全的, 你不需要考虑指针安全性。

(4) 下面的函数对 Union 进行递归操作:

```
long sum(union ele *up) {
    if (up == NULL) {
        return 0;
    }
    if (up->e2.next == NULL) {
        return up->e1.f(0);
    }
    return up->e2.x + sum(up->e2.next);
}
```

编译器为 `sum` 生成如下的汇编代码：

```
sum:
    testq    %rdi, %rdi
    je       ___I___
    pushq    %rbx
    movq     %rdi, %rax
    movq     8(%rdi), ___II___
    testq    %rdi, %rdi
    je       ___III___
    movq     (%rax), ___IV___
    call     sum
    addq     %rbx, %rax
.L1:
    popq     %rbx
    ret
.L9:
    call     *(%rax)
    jmp      .L1
.L4:
    movl     $0, %eax
    ret
```

根据 C 代码，补全汇编代码。（8 分）

- I. `je` \_\_\_\_\_
- II. `movq 8(%rdi),` \_\_\_\_\_
- III. `je` \_\_\_\_\_
- IV. `movq (%rax),` \_\_\_\_\_

## 五、绝知此事要躬行

（共 24 分）Hello! 为了检验你是否认真完成了第二章的 Bomb Lab，考试中我们制作了一个只有 2 个阶段的 Mini Bomb Lab，整个题目的完成流程与你完成 Bomb Lab 的流程类似。我们在实验桌面为你提供了 `bomb` 可执行程序 and `bomb.c`。如果你对实验流程还不熟悉，可参考实验桌面提供的《Bomb Lab 实验指导书》和《GDB 速查》。请将每个 Phase 的通关输入填写至下面的空中。考试过程中炸弹爆炸不扣分，你可以通过 `bomb` 可执行程序来验证你的答案是否正确。

- Phase 1 的通关输入为\_\_\_\_\_（12 分）
- Phase 2 的通关输入为\_\_\_\_\_（12 分）

## 六、留取丹心照汗青

(共 8 分) 考虑下面的汇编代码:

```
fun:
    movq    (%rdi), %rax
    testq   %rax, %rax
    je      .L2
    cmpq    $23, %rax
    je      ___I___
    movl    $0, %eax
    ret

.L2:
    movq    $100, 16(%rdi)
    movq    $0, 8(%rdi)

.L5:
    addq    %rax, 16(%rdi)
    movq    ___II___
    movl    $1, %eax
    ___III___

.L3:
    movq    8(%rdi), %rax
    addq    $1, %rax
    movq    %rax, 8(%rdi)
    cmpq    ___IV___
    jle     .L5
    movq    $-1, (%rdi)
    movl    $0, %eax
    ret
```

下面的 main() 函数调用上面的 fun() 函数：

```
#include <stdio.h>
typedef struct {
    long a;
    long b;
    long c;
} S;

int main() {
    S p = {0,0,0};
    fun(&p);
    printf("%d,%d,%d\n",p.a,p.b,p.c);
    fun(&p);
    printf("%d,%d,%d\n",p.a,p.b,p.c);
    int ret = fun(&p);
    printf("%d,%d,%d\n",p.a,p.b,p.c);
    return ret;
}
```

运行上面的程序，main() 函数返回 0，且前两行打印结果如下：

```
23,0,100
23,1,101
```

(1) 补全汇编代码。(4 分)

I. je \_\_\_\_\_

II. movq \_\_\_\_\_

III. \_\_\_\_\_

IV. cmpq \_\_\_\_\_

(2) 第三行的打印结果是\_\_\_\_\_ (4 分)